

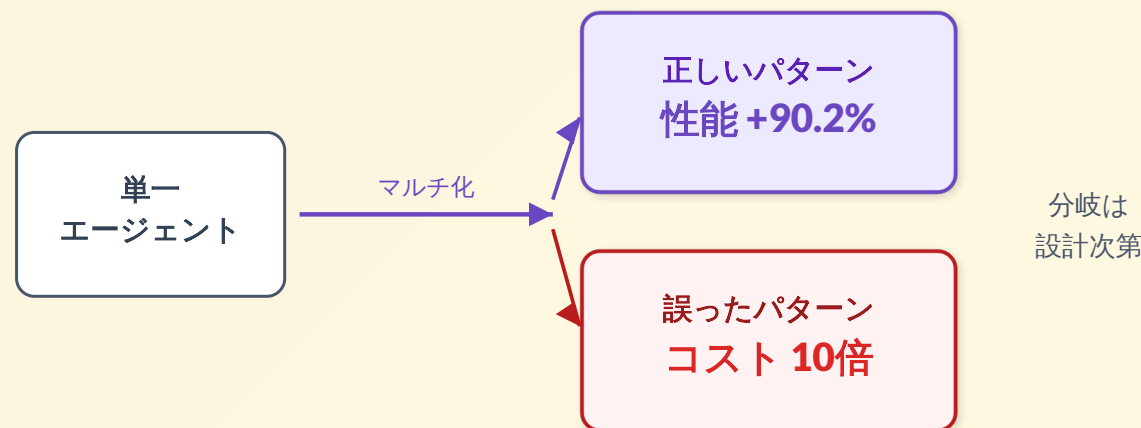
マルチエージェント設計 2026

“6つのオーケストレーションパターンと本番化の指針 / テックリーダーのための設計ガイド”

- 単一エージェントの限界を超えるための、実務に効く設計判断
- パターン選択・コスト・本番運用の観点で意思決定する

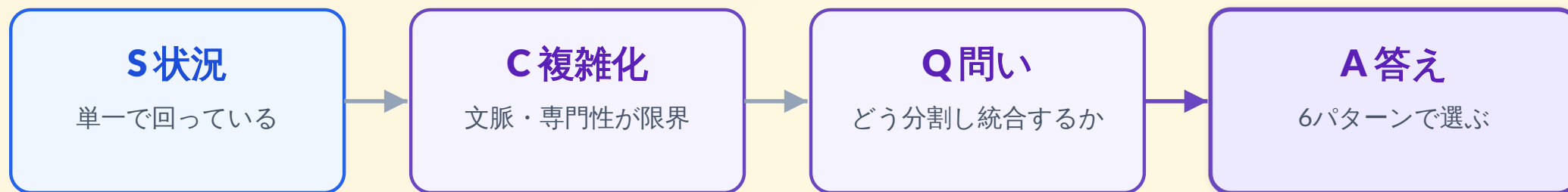
マルチ化は万能薬ではない。パターン 選択が成否を分ける

“正しいパターンなら性能+90.2%、誤ればコストは10倍に膨らむ”



本日の道筋：状況 → 複雑化 → 問い → 答えで設計判断に至る

SCQAナラティブで結論まで導く



なぜ今マルチエージェントか

- 単一エージェントの限界と、マルチ化が解く問題を整理する

単一エージェントは文脈・専門性・並列性の3点で頭打ちになる

この3点がマルチ化の動機になる

① 文脈

コンテキスト窓が
逼迫する

長い履歴で精度が
劣化・逸脱

② 専門性

分業ができない

1体に全役割を
詰め込むと指示が
曖昧化

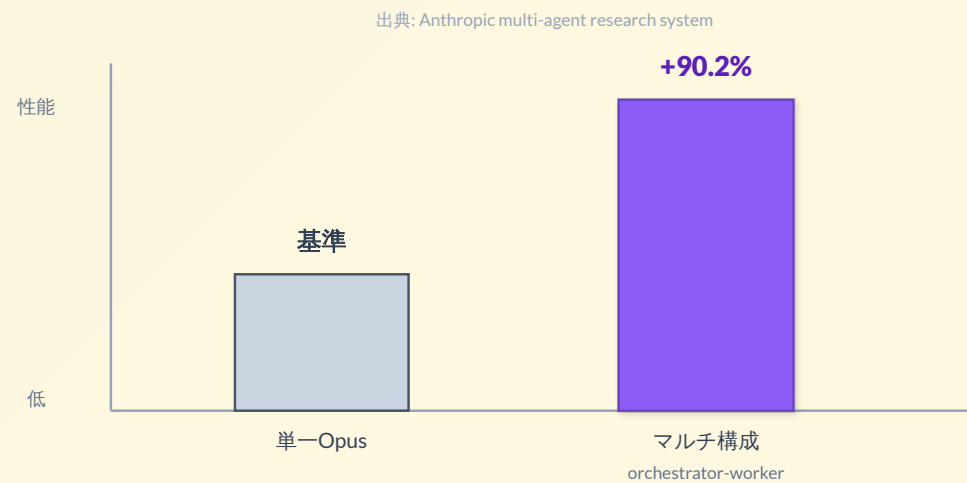
③ 並列性

逐次実行のみ

独立作業も直列に
なり待ち時間が
積み上がる

実証：マルチ構成が単一Opusを90.2%上回った

“orchestrator-worker型が広い探索タスクで単一エージェントを大きく上回る”



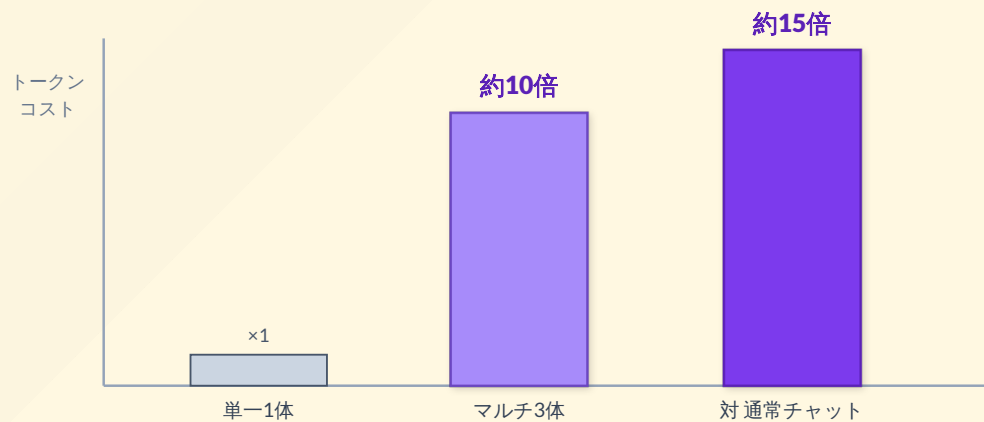
2026フレームワーク地図：用途で棲み分ける4強

“本番運用ならLangGraph、試作速度ならCrewAI、研究討論はAutoGen、軽量ハンドオフはSwarm”



だが3体構成はトークンコストが約10倍に膨らむ

“ エージェント数に対しコストは非線形に増える。通常チャット比では約15倍 ”



出典: Augment Code / Anthropic (トークンはエージェント数に非線形)

状況の要点：単一で回るうちに設計を決めておくべき理由

“ 限界に直面してから慌てて分割すると、コストと複雑さが同時に爆発する ”

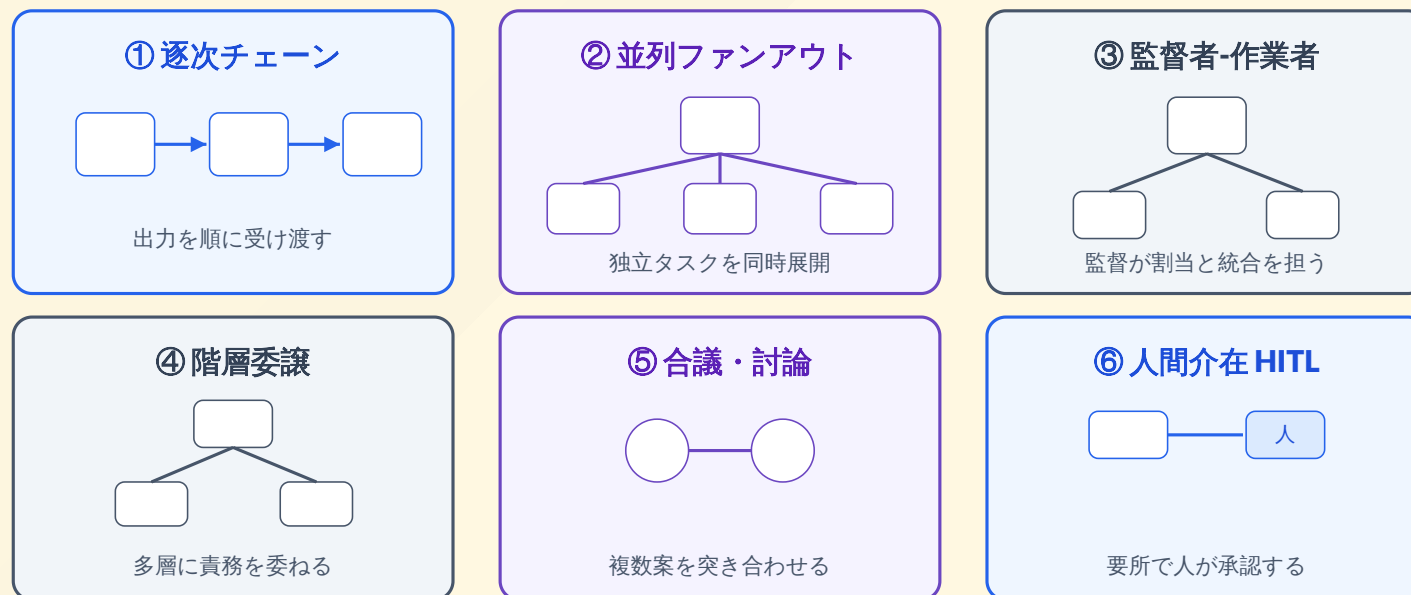
- 文脈・専門性・並列性の限界は、負荷が上がるほど同時に顕在化する
- マルチ化は性能を最大90.2%押し上げる一方、コストは10倍規模に膨らむ
- フレームワークは用途で棲み分けており、あとから乗り換える負担は大きい
- だからパターンとコスト構造を理解した上で最初の一手を選ぶ必要がある

6つのオーケストレーションパターン

- タスク特性に応じて選ぶ、6つの標準的な構成を順に見ていく

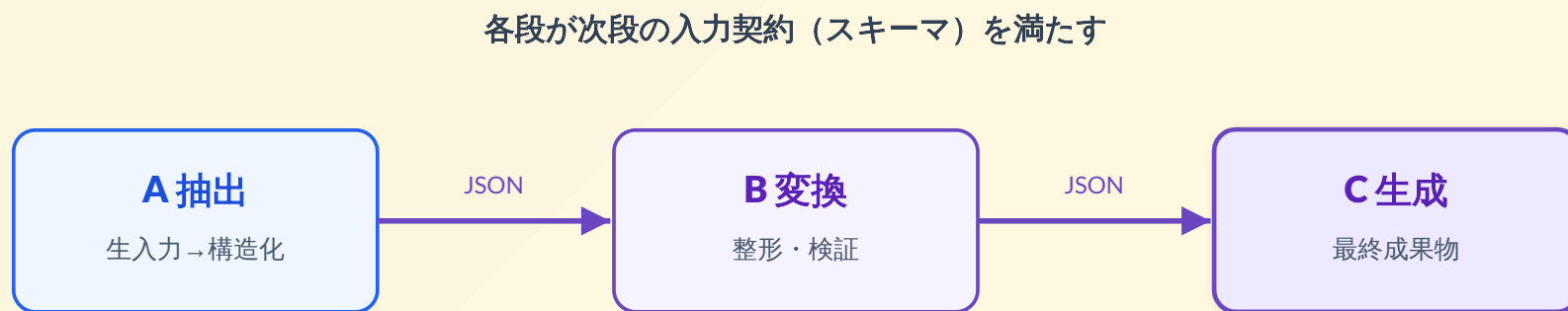
6つのパターンを1枚で俯瞰する

“ 逐次・並列・監督者-作業者・階層委譲・合議討論・人間介在の6択から選ぶ ”



①逐次チェーン：各段の構造化出力を次段へ渡す

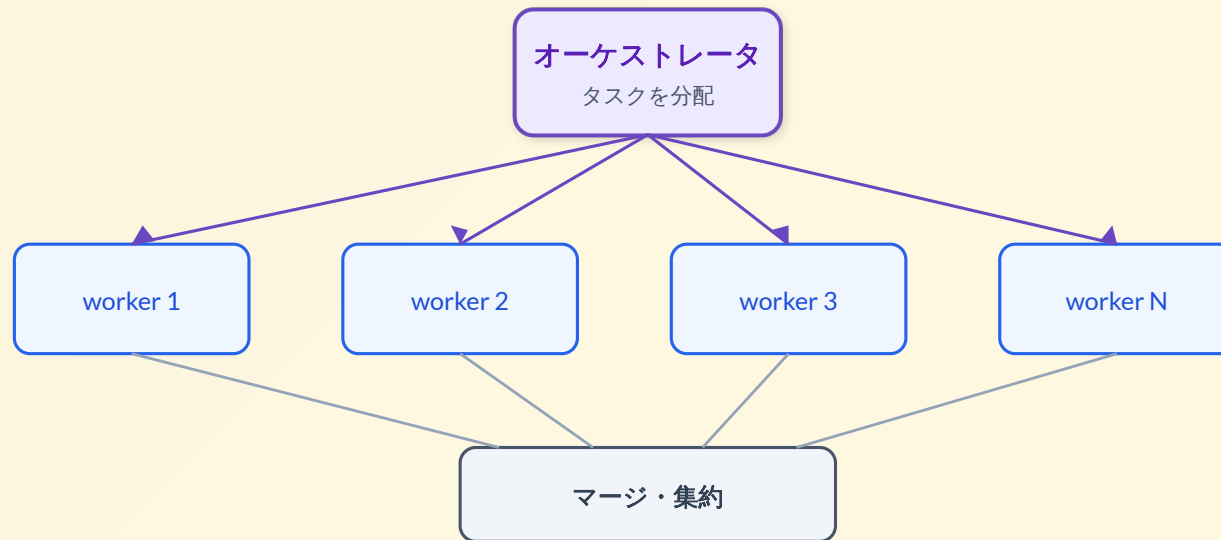
“手順が確定した処理に最適。ただし1段の失敗が全体に波及する”



向く: 手順が確定 / 向かない: 分岐や再試行が多い処理

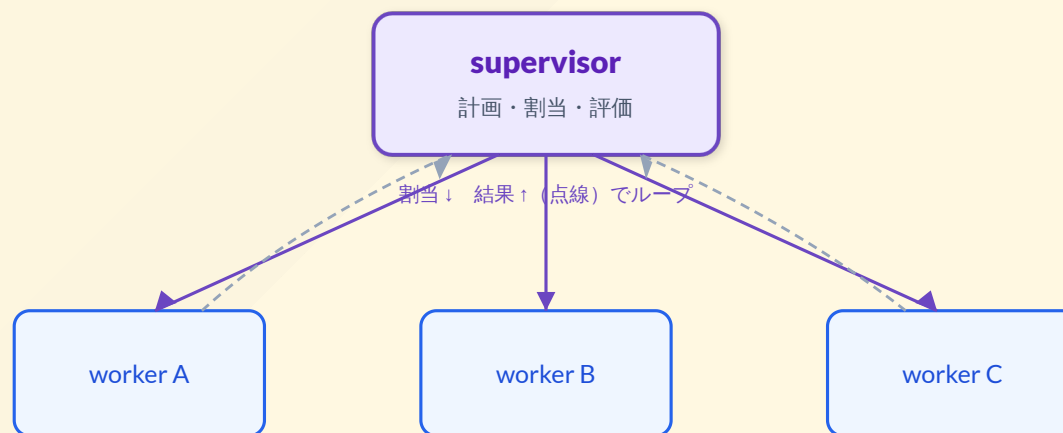
②並列ファンアウト：独立タスクを同時展開して束ねる

“調査・要約など相互依存のない作業でレイテンシを大幅短縮できる”



③監督者-作業者：最初の一歩はここから始めるべき

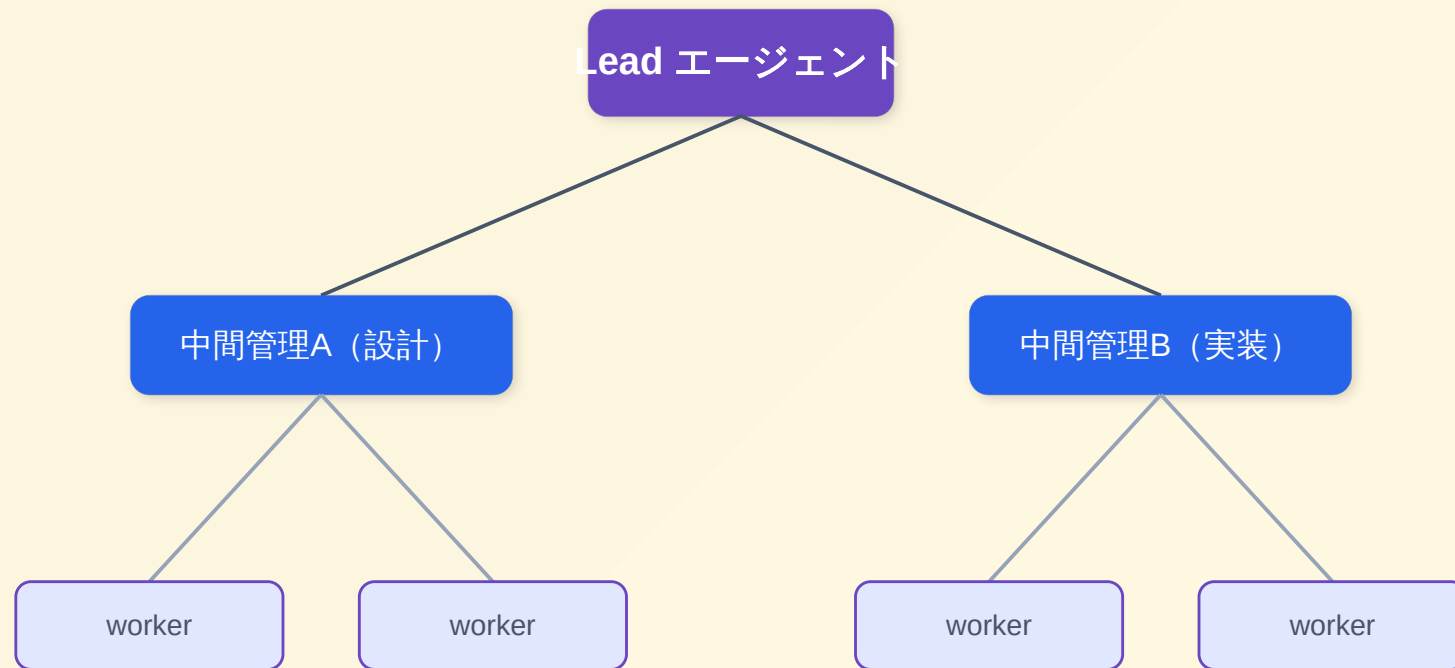
“最も広くサポートされ失敗モードが明快。マルチ化の標準的な出発点”



結果が基準未達なら supervisor が再割当（自己修正ループ）

④階層委譲は大規模タスクを木構造で分割統治する

“leadが中間層に委譲し、末端workerが並列で実務をこなす”

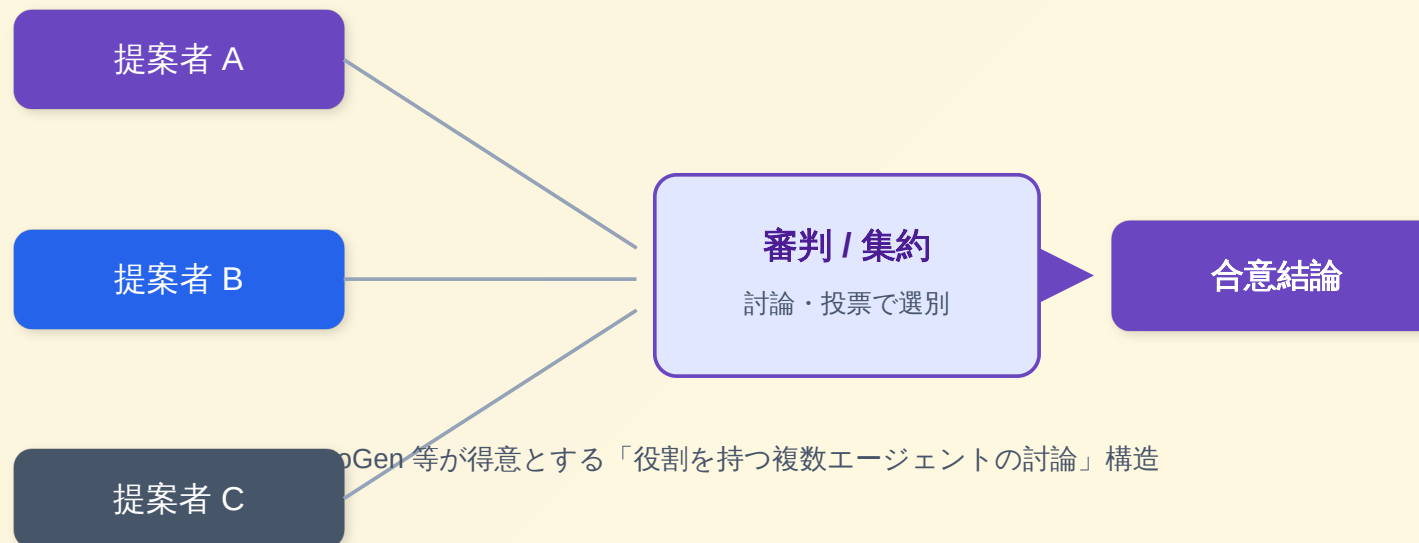


深さ方向にタスクを再帰分解し、末端で並列実行する

- 各層は「配下への委譲」だけを責務にし、末端のworkerが実作業を担う
- 巨大タスクを2～3階層に畳むことで、1エージェントの文脈窓に収める
- 各中間管理は部分成果を要約して上位へ返し、詳細は下層に閉じ込める

⑤合議・討論は複数意見を戦わせて精度を上げる

“独立した回答を審判が集約し、単体より誤りを減らす”

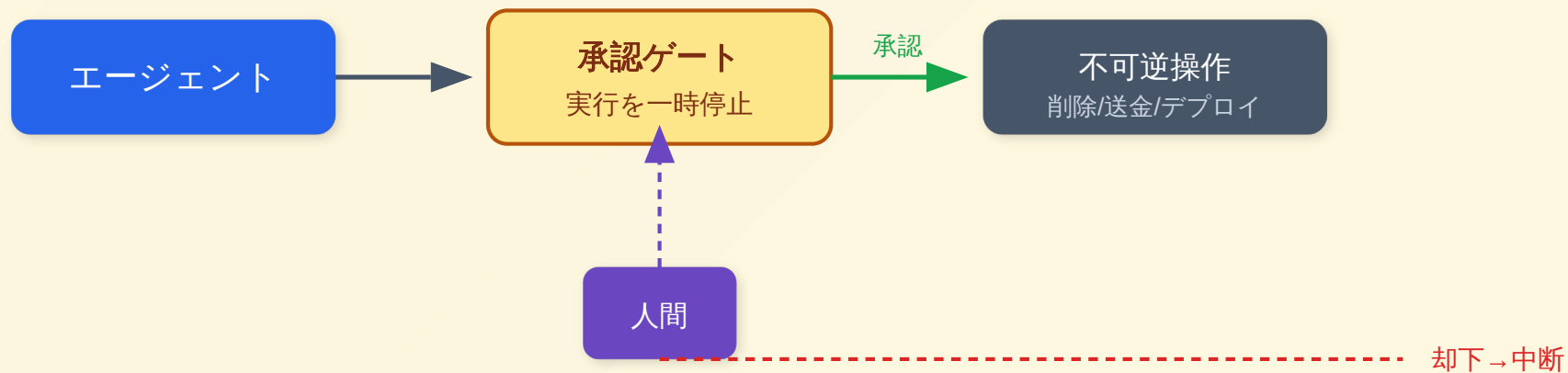


oGen 等が得意とする「役割を持つ複数エージェントの討論」構造

- 同一問いに独立回答させ、相互批判で弱い答えを淘汰する
- 審判エージェントが多数決や根拠比較で最終結論を確定する
- コストは増えるが、推論の難所や事実確認で精度が安定する

⑥人間介入は不可逆操作の直前に承認ゲートを差し込む

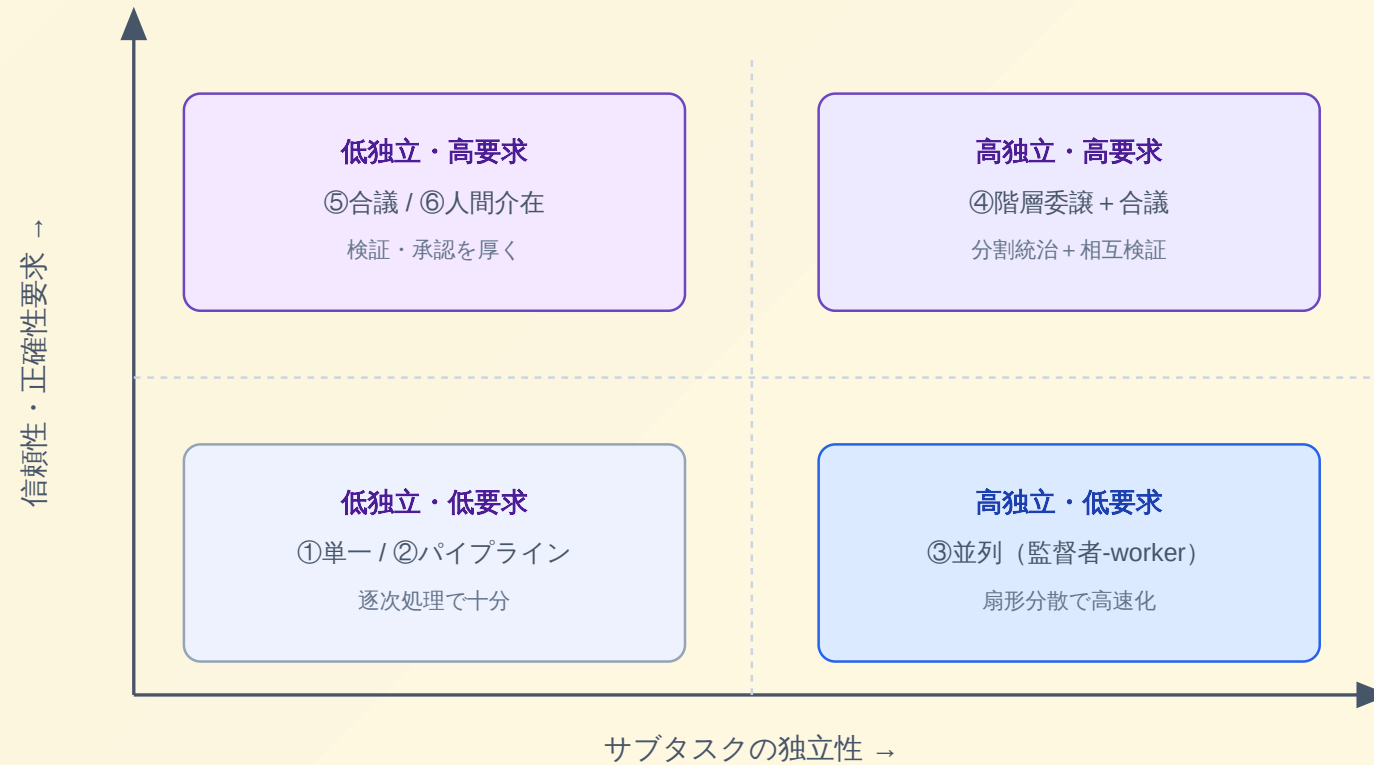
“削除・送金・デプロイの手前で必ず止め、人が最終決裁する”



- 自動化の利便と安全性を両立させ、致命的な誤操作を構造で防ぐ
- 承認対象は「取り返しがつかない副作用」に限定し、通常操作は止めない
- 承認履歴を監査ログに残し、誰が何を許可したかを追跡可能にする

独立性×信頼性要求の2軸で6パターンは選び分けられる

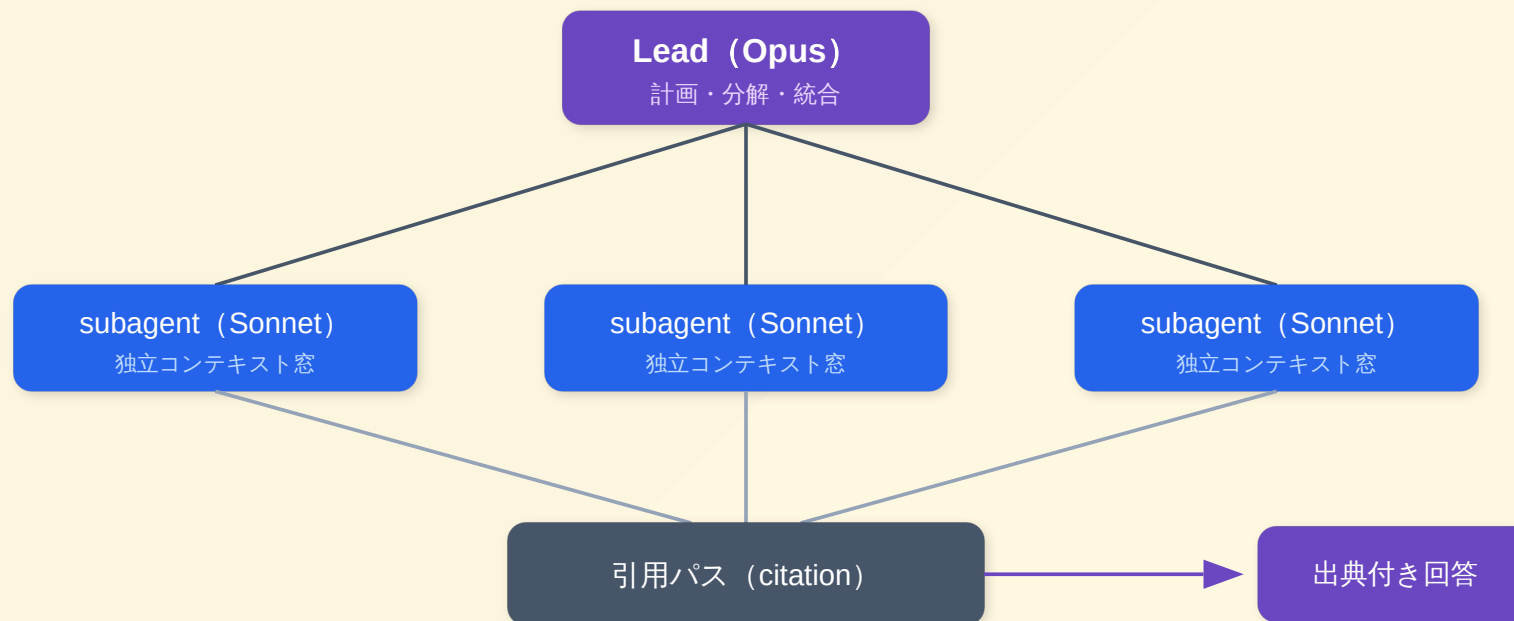
“サブタスクが独立で信頼性要求が高いほど、並列化と検証を厚くする”



- まず「サブタスクは分解できるか」で並列化の可否を判断する
- 次に「誤りの許容度」で検証層（合議・人間介在）の厚みを決める

Anthropicの実装はOpus指揮 + Sonnet並列 + 引用パスで動く

“上位モデルが計画し、下位モデルが独立文脈で並列探索する”



別建ての引用パスが各主張に根拠を紐付ける

- 指揮役に強いモデルを充て、探索は安価な並列workerに委ねる
- 各subagentは独立した文脈窓を持ち、互いの探索を汚染しない
- 引用パスを分離し、統合後の主張に出典を後付けで対応させる

迷ったら「まず監督者パターン」から始めるべき理由

“サポートが最広・失敗モードが明快・本番事例が最多の三拍子

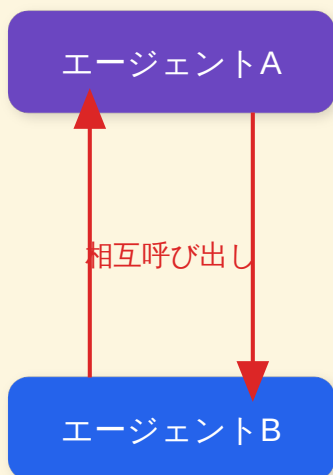
- 最も広くサポートされる：Claude Agent SDK / LangGraph / OpenAI Agents SDK / CrewAI hierarchical が標準機能として提供
- 失敗モードが明快：過剰委譲やループは反復上限・タイムアウトで機械的に抑制できる
- 本番事例が最多：orchestrator-workerは実運用の知見と対策パターンが蓄積されている
- 拡張しやすい：worker数の増減やモデル振り分けで、コストと精度を後から調整できる

過剰委譲と無限ループがコストを静かに暴走させる

“ 反復上限とタイムアウトで、増殖する呼び出しを構造的に断つ ”

”

アンチパターン



歯止め

反復上限 (max depth / max steps)

N回で強制停止し暴走を打ち切る

タイムアウト / コスト上限

実時間・トークン予算で遮断

委譲の深さ制限

末端workerは再委譲を禁止

委譲が委譲を呼び、呼び出しが指数的に増える

- 自律性を上げるほど、暴走の初期兆候はログに埋もれて見えにくい
- 上限は「性能を制限する枷」ではなく「コスト事故を防ぐ保険」と捉える

設計フェーズの結論：軸で選び、上限で守る

“パターンは独立性×信頼性で選び、自律性は必ず上限で囲う”

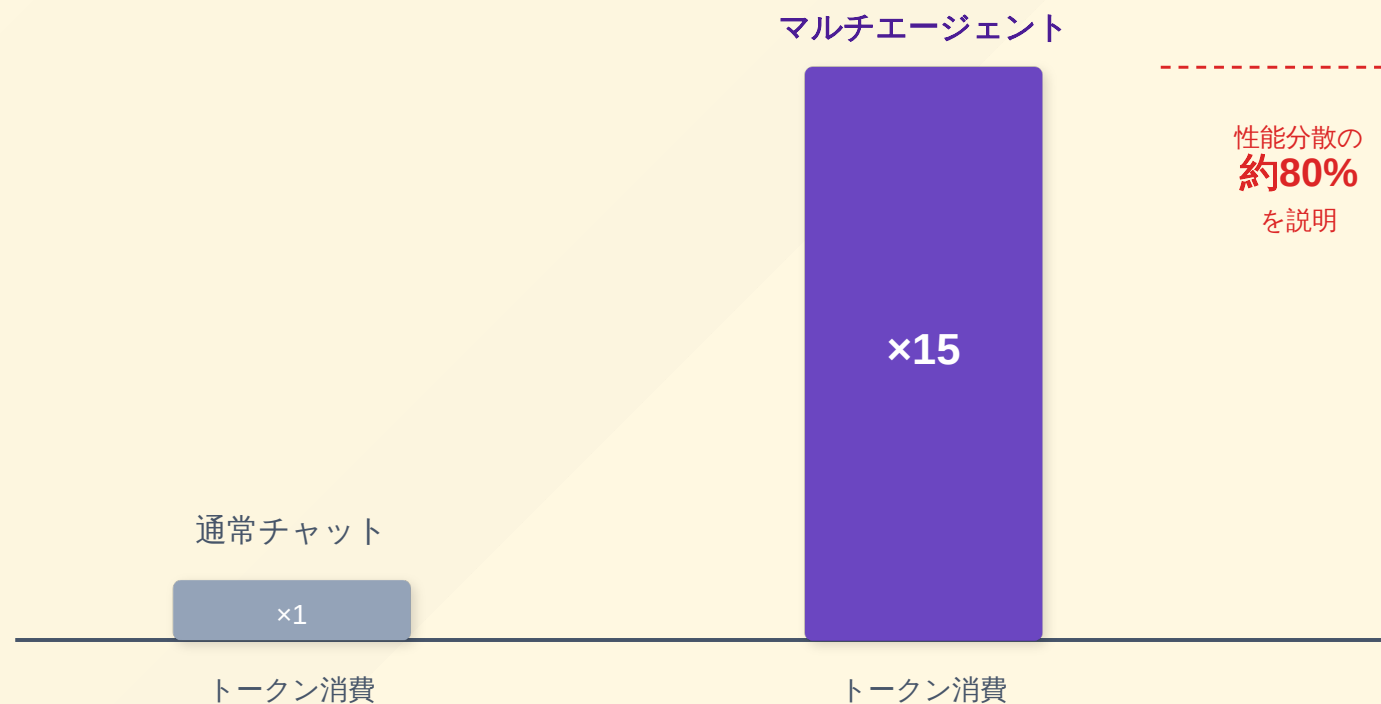
- 6パターンは工具箱：単一・パイプライン・並列・階層委譲・合議・人間介在を状況で使い分ける
- 選択軸は2つ：サブタスクの独立性で並列可否を、信頼性要求で検証層の厚みを決める
- 既定は監督者パターン：サポート最広・失敗モード明快・事例最多で、まず試す価値が高い
- 自律性には必ず枷を：反復上限・タイムアウト・深さ制限・人間介在ゲートをセットで設計する

では、何が本番化を阻むのか

- 設計が正しくても、運用の難所を越えなければ本番には出せない
- 次のセクションで、コスト・障害連鎖・可観測性・文脈受け渡しの4つの壁を見る

本番化の難所①：トークン消費は通常チャットの約15倍に膨らむ

“ 使用量が性能分散の約80%を説明する = コスト管理が性能管理そのもの ”



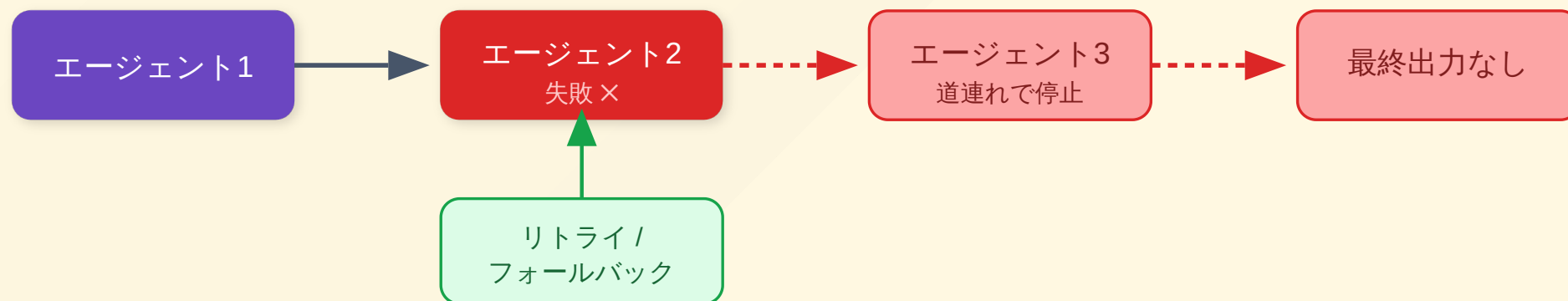
- 並列subagentと長い探索文脈が、呼び出しごとにトークンを積み上げる（出典: Anthropic）
- 費用対効果が見合うのは、高価値・並列化可能なタスクに限られる
- トークン予算を先に決め、それを制約として設計に織り込む

本番化の難所②：1体の失敗が全体を止める障害連鎖が起きる

“各ホップにリトライとフォールバックを置き、連鎖を局所化する

39

カスケード障害：1点の失敗が下流を巻き込む



- 直列の依存が長いほど、1体の失敗が全経路を無効化するリスクが高まる
- 各ホップにリトライ・タイムアウト・代替経路を置き、失敗を局所化する
- 重要経路は冗長化し、部分結果でも成立する設計に寄せる

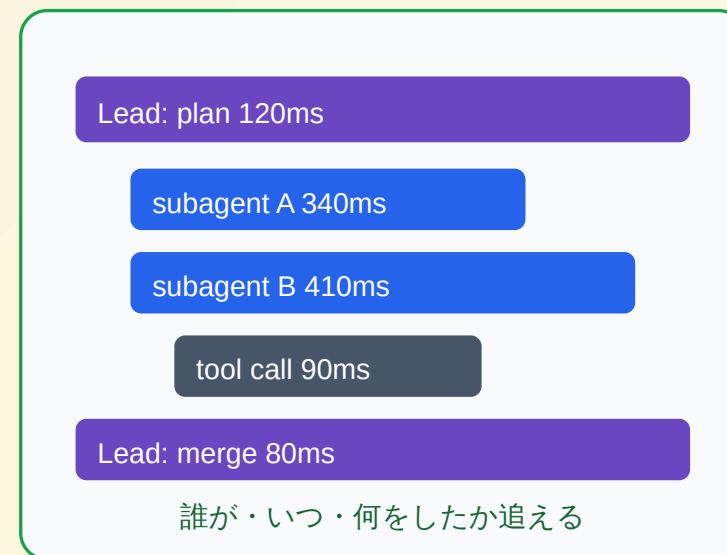
本番化の難所③：状態と可観測性の欠如で追跡不能に陥る

“分散トレースと構造化ログで、各エージェントの状態を可視化する

トレースなし = ブラックボックス



分散トレースあり



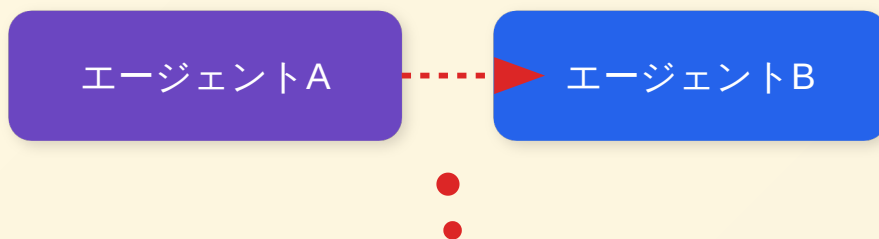
- エージェント間で状態が分散し、フレームワークによっては可観測性が弱い (例: CrewAI)
- 共通のトレースID・構造化ログで、階層をまたいだ因果を1本の時系列に繋ぐ
- コスト・レイテンシ・失敗をエージェント単位で計測し、性能分散の原因を特定する

本番化の難所④：文脈受け渡しの断絶が回答品質を崩す

“handoffを構造化し、必要な文脈を明示的に引き継ぐ”

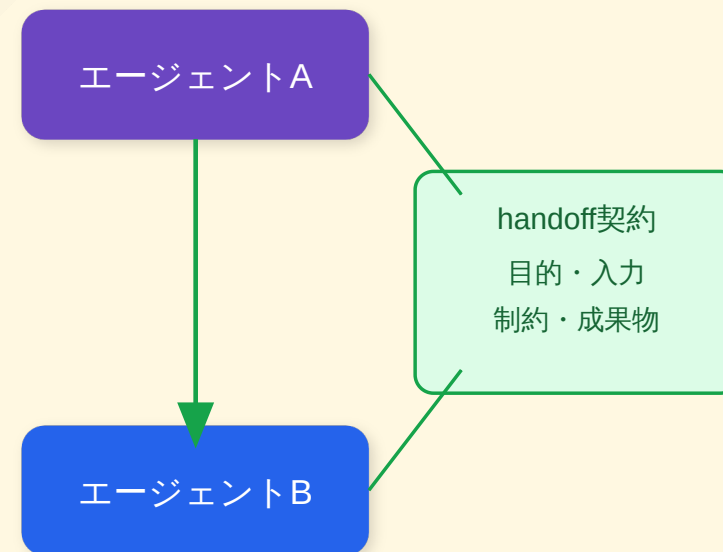
39

非構造的な引き継ぎ



前提・制約・意図が欠落 → 誤答

構造化handoff



- 自然言語だけの引き継ぎは、前提や制約が暗黙化して下流で欠落する
- 目的・入力・制約・期待成果物をスキーマ化し、handoffを契約として渡す
- 受け手が不足文脈を検知し、引き継ぎ元に問い返せる経路を用意する

本番化を阻む難所は4つに集約できる

“コスト複利・障害連鎖・可観測性欠如・文脈断絶を影響度×対処難度で俯瞰する”



難所は「設計で先回り」すれば怖くない

“ 4つの難所はいずれも後付けでは間に合わず、初期設計に組み込むことで初めて封じられる ”

- コスト複利：反復上限・トークン予算を最初から設定し、暴走を構造で止める
- 障害連鎖：エージェントを疎結合に保ち、1体の失敗を局所化する
- 可観測性欠如：トレース収集をPoC段階から標準装備にする
- 文脈断絶：エージェント間の受け渡しを明示的な契約（入出力仕様）で固める

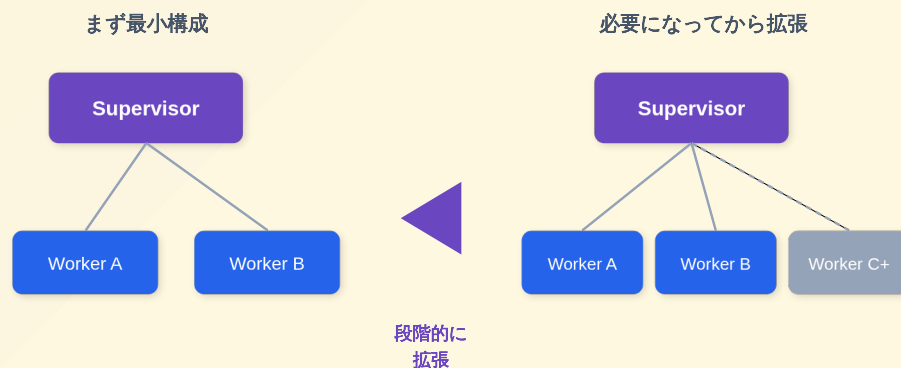
テックリーダーの設計指針

- 難所を封じる5つの実践指針と、選定・導入の型

指針①：監督者パターンから最小構成で始める

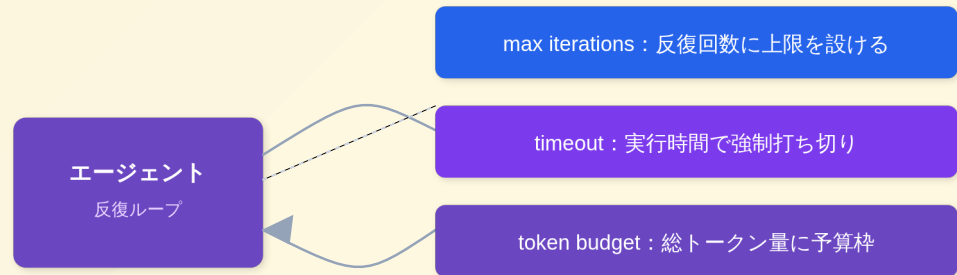
“ *supervisor+2worker*の最小構成で価値を検証し、必要に応じて段階的に拡張する

”



指針②：反復上限とタイムアウトでコストを封じる

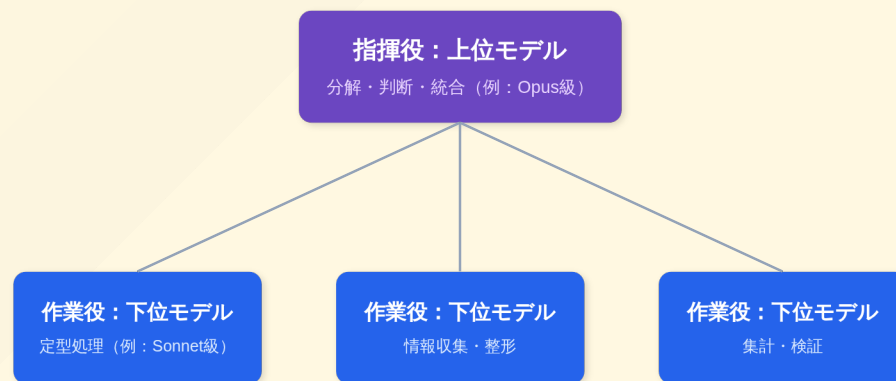
“ *max iterations · timeout · トークン予算の3つのガードレールがコスト暴走を構造的に止める* ”



3つのガードレールが揃って初めて暴走を止められる

指針③：モデルを階層で振り分けてコストを最適化する

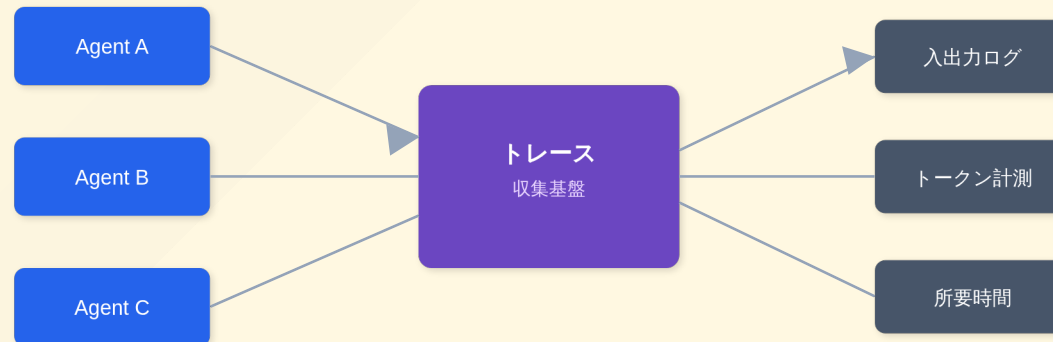
“指揮役に上位モデル・作業役に下位モデルを割り当て、品質を保ちつつコストを桁で下げる”



機械判定できる作業ほど下位モデルへ落とす

指針④：可観測性は最初からトレースで組み込む

“各エージェントの入出力・トークン・所要時間を収集し、失敗を後から追えるようにする”



指針⑤：15倍コストを吸収できる高価値タスクに絞る

“マルチエージェントはコストが跳ね上がるため、単価が高い判断集約タスクにのみ投下する”

吸収できる (投下する)

法務デューデリジェンス

競合調査・市場分析

文献レビュー・技術調査

高単価×判断集約 = 15倍を回収できる

吸収できない (避ける)

消費者向けQ&A

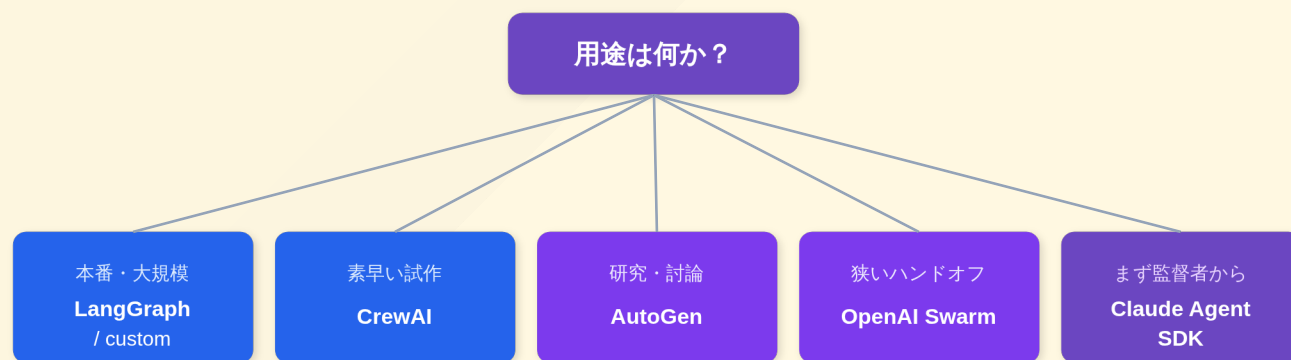
定型FAQ・単純検索

大量・低単価な処理

低単価×大量 = 単一エージェントで十分

用途からフレームワークを1枚で選び分ける

“ 本番規模・試作・研究・ハンドオフ・監督者起点で推奨が分かれる ”



迷ったら Claude Agent SDK の監督者パターンから始めるのが安全

90日で「試す → 型化 → 拡大」を回し切る

“PoCで価値を確かめ、ガードレールと可観測性で型にし、高価値タスクへ広げる”



本日の要点は3行に集約できる

1 パターン選択が成否を分ける

2 監督者パターンの最小構成から始める

3 コストと可観測性を設計に組み込む

出典・参考リンク

- How we built our multi-agent research system — Anthropic: <https://www.anthropic.com/engineering/multi-agent-research-system>
- Multi-Agent Orchestration: 5 Patterns That Work in 2026 — Digital Applied: <https://www.digitalapplied.com/blog/multi-agent-orchestration-5-patterns-that-work>
- Multi-Agent Orchestration Frameworks 2026 — Presenc AI: <https://presenc.ai/research/multi-agent-orchestration-frameworks-2026>
- Multi-Agent Cost Compounding — Augment Code: <https://www.augmentcode.com/guides/multi-agent-cost-compounding>

エージェントを「チーム」として設計 する

- 個々のモデル性能ではなく、役割分担・ガードレール・可観測性という『チーム設計』が本番の成否を決める